Question 1

Part (a)

```
import pandas as pd
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df1
```

	Even	Odd
0	2	1
1	4	3
2	6	5

Next steps: Generate code with df1 New interactive sheet

Indexing with iloc

df2 is an entirely new data frame that is a copy of df1. I came to that conclusion is because of the final output of df1 and df2. The change in df2 did not reflect on df1

```
# Indexing with iloc
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df2 = df1.iloc[[0, 1], [0, 1]]
df2.iloc[0, 0] = 0
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     2    1
1     4    3
2     6    5
```

Slicing

df2 is is really just a “view” of df1. I came to that conclusion is because of the final output of df1 and df2. The change in df2 did reflect on df1. The value at the index [0,0] for both df1 and df2 changed to 0.

```
# Slicing
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df2 = df1[0:2]
df2.iloc[0, 0] = 0
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     0    1
1     4    3
2     6    5
/tmp/ipykernel_24631/2128375444.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
  df2.iloc[0, 0] = 0
```

Boolean Expression

df2 is an entirely new data frame that is a copy of df1. I came to that conclusion is because of the final output of df1 and df2. The change in df2 did not reflect on df1

```
# Boolean Expression
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df2 = df1[df1.Even < 6]
df2.iloc[0, 0] = 0
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     2    1
1     4    3
2     6    5
/tmp/ipykernel_24631/1652574861.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
  df2.iloc[0, 0] = 0
```

Pandas Methods

df2 is an entirely new data frame that is a copy of df1. I came to that conclusion is because of the final output of df1 and df2. The change in df2 did not reflect on df1

```
# Pandas Methods
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df2 = (df1
  .query('Even < 6')
  .assign(Even = [0, 4])
)
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     2    1
1     4    3
2     6    5
```

Part (b)

After adding the copy method for slicing ,when i made df2, the "SettingWithCopyWarning" was no longer showing and df2 is now an independent copy. The changes in df2 did not reflect on df1.

```
# Slicing
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
df2 = df1[0:2].copy()
df2.iloc[0, 0] = 0
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     2    1
1     4    3
2     6    5
```

This was the one that was false positive. df2 was already a copy and not a view. Adding the copy method only removed the "SettingWithCopyWarning" .

```
# Boolean Expression
df1 = pd.DataFrame({'Even': [2, 4, 6], 'Odd': [1, 3, 5]})
```

```
df2 = df1[df1.Even < 6].copy()
df2.iloc[0, 0] = 0
print(df2)
print(df1)
```

```
   Even  Odd
0     0    1
1     4    3
   Even  Odd
0     2    1
1     4    3
2     6    5
```

Question 2

Part (a)

```
import pandas as pd
churn = pd.read_csv("churn_modelling.csv")
churn
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActive
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	
...	...	...	...	...	...	...	...	...	...	...	...	
9995	9996	15606229	Obijiaku	771	France	Male	39	5	0.00	2	1	
9996	9997	15569892	Johnstone	516	France	Male	35	10	57369.61	1	1	
9997	9998	15584532	Liu	709	France	Female	36	7	0.00	1	0	
9998	9999	15682355	Sabbatini	772	Germany	Male	42	3	75075.31	2	1	
9999	10000	15628319	Walker	792	France	Female	28	4	130142.79	1	1	

10000 rows × 14 columns

Next steps: Generate code with churn New interactive sheet

Part(b)

i chose to manually write down the columns i want.

```
churn = churn[["CustomerId", "CreditScore", "Geography", "Tenure",
               "Balance", "NumOfProducts", "HasCrCard",
               "IsActiveMember", "EstimatedSalary", "Exited"]]
churn
```

	CustomerId	CreditScore	Geography	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
0	15634602	619	France	2	0.00	1	1	1	101348.88	1
1	15647311	608	Spain	1	83807.86	1	0	1	112542.58	0
2	15619304	502	France	8	159660.80	3	1	0	113931.57	1
3	15701354	699	France	1	0.00	2	0	0	93826.63	0
4	15737888	850	Spain	2	125510.82	1	1	1	79084.10	0
...	...	...	...	...	...	...	...	...	...	...
9995	15606229	771	France	5	0.00	2	1	0	96270.64	0
9996	15569892	516	France	10	57369.61	1	1	1	101699.77	0
9997	15584532	709	France	7	0.00	1	0	1	42085.58	1
9998	15682355	772	Germany	3	75075.31	2	1	0	92888.52	1
9999	15628319	792	France	4	130142.79	1	1	0	38190.78	0

10000 rows × 10 columns

Next steps: Generate code with churn New interactive sheet

Part (c)

I found isnull from the book we use in class (Python Data Science Handbook) not from the class. isnull(): Generate a boolean mask indicating missing values then used the any method on it

```
churn.isnull().any()
```

	0
CustomerId	False
CreditScore	False
Geography	False
Tenure	False
Balance	False
NumOfProducts	False
HasCrCard	False
IsActiveMember	False
EstimatedSalary	False
Exited	False

dtype: bool

Part (d)

I took the churn DataFrame, kept only the Geography, Exited, and Balance columns, and then grouped the rows by both Geography and Exited. I then calculated the mean Balance for each group and used .reset_index() to turn the group labels back into normal columns.

```
indiv_mean = (
    churn
    .loc[:, ['Geography', 'Exited', 'Balance']]
    .groupby(['Geography', 'Exited'])
    .mean()
    .reset_index()
)

indiv_mean
```

	Geography	Exited	Balance
0	France	0	60339.275678
1	France	1	71192.795728
2	Germany	0	119427.106696
3	Germany	1	120361.075590
4	Spain	0	59678.070470
5	Spain	1	72513.352446

Next steps: Generate code with `indiv_mean` New interactive sheet

Part (e)

```
newcust = pd.read_csv("new_customers.csv")
merge_cust_churn = pd.merge(newcust, churn, on="CustomerId", how="left")
cust_never_left = merge_cust_churn[merge_cust_churn["Exited"] != 0]
# simple way
total_cust_returned = len(cust_never_left[cust_never_left["Exited"] == 1])
percent_cust_returned = 100 * total_cust_returned / len(cust_never_left)
print('simple way', percent_cust_returned)
# fancy way
percent_fancy_return = (
    cust_never_left
    .assign(total_cust_returned = cust_never_left["Exited"] == 1)
    .loc[:, ["total_cust_returned"]]
    .mean()
    * 100
)
print('fancy way', percent_fancy_return)
```

```
simple way 0.8585445625511039
fancy way total_cust_returned    0.858545
dtype: float64
```

Summary

I merged the old churn data frame with the new customers by customer ID, removed rows of new customers where Exited was equal to zero, and calculated the percentage of those customers with Exited = 1 in two ways: once by counting manually and once by using a boolean column and .mean().

Question 3

I uploaded the files and then created three new columns for freshmen and juniors that add together each student's physical, intellectual, and financial scores. I obtained a new data frame with the combined scores for both freshmen and juniors individually. Then, I joined both sets of scores into a new data frame (stu_wellness) that can compare individuals' scores between the first and third years. At the end, I created the difference columns to compare them and produced the box plot.

```
freshman = pd.read_csv("firstyear.csv")
junior = pd.read_csv("junior.csv")

freshman["Phys"] = (
    freshman["Phys0"] + freshman["Phys1"] + freshman["Phys2"] + freshman["Phys3"] +
    freshman["Phys4"] + freshman["Phys5"] + freshman["Phys6"] + freshman["Phys7"] +
    freshman["Phys8"] + freshman["Phys9"]
)

freshman["Int"] = (
    freshman["Int0"] + freshman["Int1"] + freshman["Int2"] + freshman["Int3"] +
    freshman["Int4"] + freshman["Int5"] + freshman["Int6"] + freshman["Int7"] +
    freshman["Int8"] + freshman["Int9"]
)

freshman["Fin"] = (
```

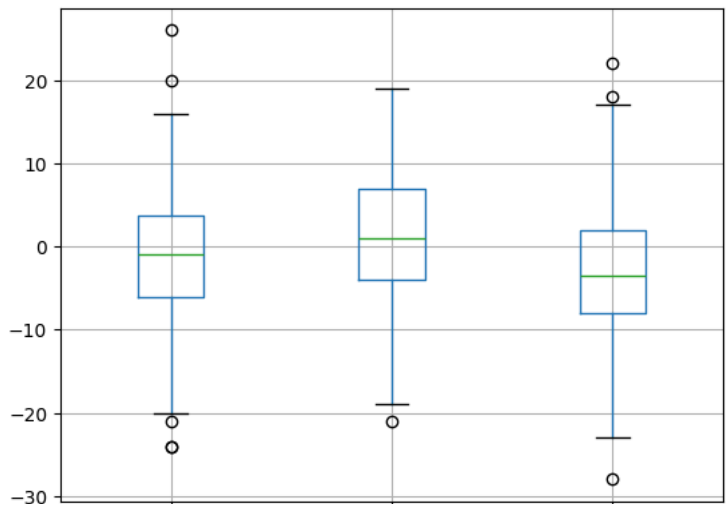
```
    freshman["Fin0"] + freshman["Fin1"] + freshman["Fin2"] + freshman["Fin3"] +
    freshman["Fin4"] + freshman["Fin5"] + freshman["Fin6"] + freshman["Fin7"] +
    freshman["Fin8"] + freshman["Fin9"]
)

freshman_combined_scores = freshman[["ID", "Phys", "Int", "Fin"]]

junior["Phys"] = (
    junior["Phys0"] + junior["Phys1"] + junior["Phys2"] + junior["Phys3"] +
    junior["Phys4"] + junior["Phys5"] + junior["Phys6"] + junior["Phys7"] +
    junior["Phys8"] + junior["Phys9"]
)

junior["Int"] = (
    junior["Int0"] + junior["Int1"] + junior["Int2"] + junior["Int3"] +
    junior["Int4"] + junior["Int5"] + junior["Int6"] + junior["Int7"] +
    junior["Int8"] + junior["Int9"]
)

junior["Fin"] = (
    junior["Fin0"] + junior["Fin1"] + junior["Fin2"] + junior["Fin3"] +
    junior["Fin4"] + junior["Fin5"] + junior["Fin6"] + junior["Fin7"] +
    junior["Fin8"] + junior["Fin9"]
)

junior_combined_scores = junior[["ID", "Phys", "Int", "Fin"]]

stu_wellness = pd.merge(
    freshman_combined_scores,
    junior_combined_scores,
    on="ID",
    how="inner"
)

stu_wellness["DPhys"] = stu_wellness["Phys_y"] - stu_wellness["Phys_x"]
stu_wellness["DInt"] = stu_wellness["Int_y"] - stu_wellness["Int_x"]
stu_wellness["DFin"] = stu_wellness["Fin_y"] - stu_wellness["Fin_x"]

diffs = stu_wellness[["DPhys", "DInt", "DFin"]]
diffs
```

	DPhys	DInt	DFin
0	-13	1	-11
1	-2	-7	-2
2	1	6	-11
3	-9	9	-7
4	-11	-4	-3
...	...	...	...
193	-11	2	4
194	4	2	-4
195	8	2	0
196	-1	-1	-16
197	0	-1	5

198 rows × 3 columns

Next steps: Generate code with `diffs` New interactive sheet

After getting the box plot, I tried to find the median of each. The median for financial wellness was the lowest, followed by physical, while intellectual was the highest and positive. This shows that as students move from the first year to the third year, their financial and physical wellness decrease, but their intellectual wellness increases. Because financial deviates a lot from 0, it the wellness dimension that changes most

```
diffs.boxplot()
```

```
<Axes: >
```